A Survey of AI-Assisted Reverse Engineering and Binary Analysis

Josiah Schmitz
Wright State University
schmitz.74@wright.edu

***Abstract*—Reverse engineering is a critical technique that is useful in many aspects of computer technology, such as analyzing malware or identifying vulnerbilities, but it is a very difficult task due to compiler optimizations, binary stripping, or lack of semantic information in the code. Recently, significant progress in generative AI, particularlly LLMs, has provided an alternate way to automate and enhance reverse engineering operations. This survey covers recent papers (2020-2026) related to the use of artificial intelligence in reverse engineering.**

The selected works include systems for decompilation (e.g., LLM4Decompile, SALT4Decompile), symbol recovery (e.g., ReCopilot), and evaluation frameworks (e.g., REBench, DecompileBench, BinMetric), as well as broader surveys and supporting techniques. This paper divides the covered material into several categories depending on the task, model architecture, data representation, and evaluation approach and compares the various papers' findings with each other on the basis of effectiveness, methodology, and applications. Moreover, the various advantages and disadvantages of existing methodologies, such as hallucinations, architecture-independent models, and dataset dependency are analyzed by their individual severity.

I. Introduction

Reverse engineering is the process of analyzing compiled code to discover low-level structural, functional, and behavioral information about the code without access to the original source code. It's very useful in fields such as cybersecurity, malware analysis, and digital forensics. However, reverse engineering can be very difficult due to the fact that compilation often removes a lot of the high-level semantic information found in source code. Compiler optimizations, stripped debugging symbols, and code obfuscation further complicate the process of trying to reverse engineer source code.

Traditional reverse engineering tools, including disassemblers and decompilers, have significantly improved software analysis but are often limited when trying to reconstruct accurate representations of binaries. While these tools can sometimes recover control flow and produce readable pseudocode, they often struggle to infer variable names, function names, or data types. As software systems continue to increase in complexity, researchers have begun exploring artificial intelligence to help automate binary analysis.

Recent advances in generative artificial intelligence, including large language models (LLMs), have assisted the process of automating reverse engineering. One of the biggest benefits of LLMs is that they can use patterns learned from their training data to infer semantic patterns that are not explicitly defined in binary code. This has led to the development and research of LLM-based systems capable of binary decompilation, function and variable name recovery, vulnerability analysis, and interactive use assistance for reverse engineers. At the same time, researchers have introduced standardized benchmarks and datasets to objectively evaluate these increasingly sophisticated models.

Even though AI-assisted reverse engineering is a rapidly growing field, it has a significant amount of diversity in the design and implementation of its systems. Some studies focus on developing specialized language models, while others work on creating benchmarks, generating robust datasets, or using other deep learning architectures like Vision Transformers. Understanding the strengths and weaknesses of these varying approaches is very useful for guiding future research and system designs.

This survey reviews ten related papers published between 2024 and 2026 that discuss the potential application of artificial intelligence to reverse engineering and binary analysis. The surveyed works include specialized LLM-based decompilation systems, AI-assisted reverse engineering assistants, benchmark frameworks, software security surveys, and supporting machine learning techniques. The papers are compared according to their target applications, model architectures, data representations, evaluation methodologies, and reported performance. Finally, this survey identifies some common limitations of AI-assisted reverse engineering and discusses future work for other researchers in the field.

II. Related Work

A. AI-assisted Reverse Engineering

Many researchers have investigated if AI, specifically LLMs, can be useful for general reverse engineering tasks. G. Chen et al. used an LLM known as ReCopilot that was designed specifically for binary analysis and reverse engineering tasks. It has three main training stages to it: continue pretraining (CPT), supervised fine-tuning (SFT), and direct preference optimization (DPO). CPT is used to gain the core knowledge and language features from the raw data. SFT adapts the model to its predefined tasks after it has been pre-trained. DPO uses reinforcement from user preferences to better shape the LLM's final output. The authors developed a multi-task benchmark to evaluate ReCopilot on its success with tasks such as function name recovery, variable name

recovery, variable type recovery, decompilation and more. They also used several benchmarks to evaluate the more general capabilities of the model that were less focused on binary analysis, such as mathematical reasoning and instruction following. The authors then compared ReCopilot's binary analysis results to those of other LLMs, including DeepSeek and LLM4Decompile. This comparison showed that ReCopilot succeeded compared to other models on most tasks by a factor of 5-14%. This success was most evident in the variable name recovery task. The authors cover several potential weaknesses with ReCopilot, like its lack of supported programming languages and binary representations, and suggests improvements to its learning methods to improve these areas.

Unlike ReCopilot, which is a specifically binary analysis-focused LLM, S. Pordanesh's and B. Tan's study used GPT-4, a more general-use LLM, for binary reverse engineering. They tested GPT-4's ability to interpret both human-written and decompiled code through a series of experiments divided into two phases. One phase focuses on the LLM's basic ability to understand code through tasks such as code explanation, function and variable renaming, and identifying program characteristics. The other phase tests GPT-4's performance on reverse engineering scenarios involving complex malware analysis. The authors assess the quality of GPT-4's responses by comparing the results to given tutorials and code explanations using both Python scripting and manual confirmation. The authors found that the model performs well on tasks such as function identification and network related programs, but struggled on complex technical analyses that require binary-specific knowledge with stealth techniques having the highest error rate of any task type. They conclude that while GPT-4 has considerable potential as an assistant for reverse engineering, it lacks the specialized training necessary for accurate independent binary analysis. The paper also discusses limitations in evaluation methodology and dataset availability, suggesting that future research should focus on developing domain-specific models and more sophisticated and standardized benchmarks for binary reverse engineering.

B. Binary Decompilation

Another aspect of reverse engineering that researchers are testing LLMs on is that of binary decompilation, the process of converting binary code to high-level source code. H. Tan et al. propose LLM4Decompile, an open-source group of LLMs designed specifically for binary code decompilation. The authors argue that traditional decompilers, Ghidra being a notable example, often produce pseudocode that is difficult to understand or execute correctly for a human user due to the loss of significant amounts of semantic information during the compilation process. In an attempt to address this, they train models on a total of approximately 7 billion tokens consisting of assembly language code and C source code. The authors introduce two major LLM4Decompile tools: LLM4Decompile-End for directly translating assembly to C code and LLM4Decompile-Ref for improving pseudocode that has already been generated by Ghidra. The paper also introduces HumanEval and ExeBench, benchmarks that evaluate decompilation quality by using real-world C functions and solutions to assess the LLMs code generation abilities. The results show that LLM4Decompile significantly outperforms both GPT-4 and traditional decompilation approaches, with higher re-executability rates on both the HumanEval and ExeBench benchmarks, while the LLM4Decompile-Ref model provides an additional 16.2% performance increase over LLM4Decompile-End. The authors conclude that large language models can significantly improve binary decompilation but acknowledge that current models are limited to x86-64 binaries and would need to be expanded to support other programming languages and architectures.

A second LLM-based binary decompilation tool that Y. Wang et al. discuss in their research paper is SALT4Decompile. SALT4Decompile was created to address the problem of LLMs not recognizing arbitrary jump patterns or isolated data segments in their analysis of binary executables. SALT4Decompile attempts to solve this issue by creating a Source-level Abstract Logic Tree (SALT) from assembly code to shape the high-level logic structure of the generated code. The core recovery process that SALT uses has four steps: CFG (Control Flow Graph) extraction, instruction normalization, jumping unit identification, and tree construction. CFG extraction involves gathering all the basic blocks of the assembly code and mapping out all the control flow transitions between the blocks for better flow analysis. Instruction normalization converts absolute addresses in jump instructions to relative offsets from the function's entry point, assisting the LLM with interpreting isolated data references. Jumping unit identification is done by analyzing CFGs for loops and checking any loops for potential subgraphs that might represent other loop-related jumps. Tree construction is the final step in SALT accomplished by creating a tree where the root node is the function's address and the other nodes are the logic blocks contained within the function. SALT4Decompile is then evaluated on the basis of five main research questions which address issues such as general effectiveness of decompilation, how well SALT4Decompile assists real-world binary software, or the simple cost of using SALT4Decompile. Additionally, the authors used a user study to gauge the general helpfulness and success of SALT4Decompile as measured by the users in the study. Results showed that both existing LLM-based tools, such as LLM4Decompile, that used SALT as well as SALT4Decompile itself showed significant success on most measured categories. SALT4Decompile had a 70.4% test case pass rate, which was an improvement of 12.5% over LLM4Decompile when not using SALT. The user study presented generally positive feedback on SALT4Decompile as well. Some issues with SALT4Decompile that the authors want to improve are the tendency for it to merge identical basic blocks, which often breaks loop structure identification, and the lack of functionality to identify conditional statements as

distinct from loops.

C. Benchmarks and Evaluations

Many other researchers have begun looking into creating benchmarks and standardizing evaluations for LLM-assisted reverse engineering tasks. Won et al. propose REBench, a comprehensive benchmark that aims to provide a standardized evaluation framework for LLMs doing reverse engineering tasks on stripped binaries. The authors argue that because many previous studies have used different datasets, preprocessing techniques, and evaluation metrics, it has become difficult to compare the performance of competing approaches. To address this issue, REBench consolidates datasets that have been used in earlier reverse engineering research into a single benchmark that covers four major processor architectures and various compiler optimization levels, while also compiling binaries into a universal format. The benchmark uses byte-level stack information to automatically generate ground truth for analysis tasks such as function name recovery and type inference. The authors evaluate several commonly-used LLMs, such as CodeLlama and Deepseek-R1, using REBench and find that even the most robust models struggle with complex tasks like assembly-based function name recovery, especially when analyzing highly optimized binaries or less common processor architectures. These results show the limitations that many reverse engineering LLMs still struggle with, even when using better benchmarks and evaluation methods. The authors conclude that standardized benchmarks are essential for measuring progress in AI-assisted reverse engineering.

For another benchmarking tool, Tan et al. suggest Decompile-Bench, a large-scale dataset and benchmark designed to aid research in LLM-based binary decompilation. The authors argue that existing datasets are either too small, synthetically generated, or fail to accurately map binary functions to their original source code due to compiler optimizations. To overcome these limitations, they construct the first binary-source dataset containing approximately two million binary-source function pairs, condensed from over 100 million collected pairs compiled from GitHub projects. In addition to the dataset, the authors develop Decompile-Bench-Eval, an evaluation benchmark consisting of manually crafted binaries from HumanEval, MBPP, and binaries from recent GitHub projects. The paper also covers the three main evaluation metrics for decompilation quality which are re-executability, Relative Readability Index (R2I), and edit similarity. Experimental results on comprehensive evaluation demonstrate that fine-tuning LLM-based decompilers with Decompile-Bench improves re-executability by approximately 20% compared to previous datasets. This underscores how effective and useful Decompile-Bench is for training LLM-based decompilers. The authors conclude that the existence of a large-scale benchmark like Decompile-Bench can further the development of more accurate AI-assisted decompilation frameworks while also standardizing testing and evaluation methods for future research.

Shang et al. propose a third benchmarking tool, BinMetric, which is a comprehensive benchmark for evaluating large language models on binary code analysis tasks. The authors argue that while LLMs have demonstrated significant proficiency in complex reverse engineering tasks, comparisons between models remain difficult due to the lack of standardized evaluation datasets and metrics. To address this issue, they construct a benchmark consisting of 1,000 evaluation questions derived from 20 real-world open-source software projects involving six practical binary analysis tasks, including decompilation, code summarization, assembly instruction generation, and binary understanding. Rather than introducing a new reverse engineering model, the authors use BinMetric to evaluate several state-of-the-art LLMs, including CodeLlama, DeepSeek, and GPT-4, and analyze their strengths and weaknesses across these tasks. Their results show that while modern LLMs perform well on high-level reasoning tasks such as code summarization, they continue to struggle with low-level reverse engineering problems, particularly precise binary lifting and assembly synthesis. The authors conclude that BinMetric provides a standardized and reproducible evaluation framework that facilitates more fair comparisons between AI-assisted reverse engineering systems.

D. Supporting Research

There are also many other research articles that cover more general aspects of how LLMs can be used for the benefit of computer systems, including such areas as security or reverse engineering. Zhu et al. present ”When Software Security Meets Large Language Models: A Survey”, a review of how large language models have been applied to various software security tasks. Rather than focusing exclusively on reverse engineering, the authors examine a broad range of applications including fuzzing, unit test generation, program repair, bug reproduction, data-driven bug detection, and bug triage. The survey organizes existing research by breaking down each software security task into several stages and seeing how LLMs can help at each stage, from code understanding and generation to automated reasoning and decision making. The authors also analyze exactly how LLMs engage with the pre-processing, prompt generation, and post-processing steps in LLM usage. In addition to reviewing current techniques, the authors discuss common challenges faced by LLM-based software security systems, including hallucinations, limited domain knowledge, small evaluation datasets, and potential problems with reliability and security. The paper concludes by identifying several future research directions, such as minimizing costs and maximizing access to LLMs and training data for them, improving domain-specific training, combining LLMs with more traditional, non-AI security approaches, and creating more robust evaluation methodologies. As an early survey of LLMs in software security, this work gives better context for understanding recent advances in AI-assisted reverse engineering and how

LLMs can be especially useful in a security setting, as well as a binary analysis and reverse engineering context.

A similar survey paper that discusses security and how certain techniques, including AI, can mitigate certain risks in software is A. Adhikari's and P. Kulkarni's paper "Survey of techniques to detect common weaknesses in program binaries." This survey focuses on static techniques that can be used to detect weaknesses in software binaries. Unlike source-code vulnerability detection, binary analysis doesn't have access to high-level semantic information, which makes vulnerability detection much more difficult. The authors survey eleven papers covering state-of-the-art static techniques developed to identify their top ten CWEs in compiled binaries, which include out-of-bounds write, null pointer dereferences, and integer overflows. In addition to reviewing existing research, they evaluate the effectiveness of various binary analysis tools by testing them on standard benchmarks such as SARD and Juliet. Their evaluation reveals that many binary-level tools struggle with high false-positive and false-negative rates as well as low accuracy and limited scalability. The survey discusses how many researchers are utilizing a combination of static analysis with non-staic techniques such as symbolic execution, fuzzing, and machine learning to improve vulnerability detection. The authors conclude that while binary vulnerability detection has advanced considerably, there is still much difficulty in identifying many important software weaknesses, highlighting the need for more accurate and scalable analysis methods.

The last paper is one by W. Khan et al. titled "Compiler-Provenance Identification in Obfuscated Binaries Using Vision Transformers". Khan et al. propose a using Vision Transformers (ViTs) for compiler provenance identification in obfuscated binaries. The authors note that determining the compiler family, version, and optimization settings used to produce a binary is an important and often very difficult reverse engineering task. These determinations are extremely useful for sub tasks such as malware analysis, code clone detection, function fingerprinting, and authorship attribution, but are frequently made more challenging through the existence of obfuscation techniques applied to the binaries. This is usually because obfuscation hides many of the syntactic and semantic features used by many machine learning methods. To address this challenge, the authors transform binary executables into images and employ Vision Transformers to classify compiler provenance. This is done through a two-stage classification process that first predicts the compiler family and then determines the compiler optimization level. Experimental evaluation on two different models of Vision Transformers showed great success with over 98% classification accuracy, outperforming many other machine learning methods. This approach also achieved over 95% accuracy for optimization-level identification. The authors conclude that computer vision models can effectively identify compiler-provenance in binaries, and they also suggest doing more research by applying hyperparameter training to the same algorithms.

III. Comparison, Findings, and Insights

A. Reverse Engineering Tasks

The surveyed works cover several different reverse engineering tasks. ReCopilot and the GPT-4 approach focus on helping with binary comprehension and semantic recovery, while LLM4Decompile and SALT4Decompile specifically address source-level decompilation of binaries. REBench, BinMetric, and Decompile-Bench all serve to provide standardized benchmark and evaluation frameworks. Lastly, the compiler provenance identification study use AI for classifying compiler attributes in obfuscated binaries, and the binary weakness detection survey covers more traditional analysis techniques for identifying weaknesses in binary executables. This various papers cover most of the major approaches used by researchers for reverse engineering tasks, particularly when using AI for assistance.

B. Model Architectures

Many different LLM model architectures have been covered in the surveyed papers for the various reverse engineering tasks being studied. ReCopilot, LLM4Decompile, and SALT4Decompile are all transformer-based LLM-assisted tools that have been specifically engineered and trained for their respective tasks. GPT-4, on the other hand, is a commercially available and widely used general-purpose AI that was tested, with moderate success, in a reverse engineering landscape. When comparing these models, there is a clear advantage of having specially trained models that excel at complex binary analysis, unlike GPT-4, but GPT-4 has the major benefit of ease-of-use for a general consumer and having a larger amount of research done on the AI's capabilities. The compiler provenance identification paper takes a completely different approach by using Vision Transformers, showing that even image-based deep learning can have significant benefits in a binary analysis context. This wide spread of artificial architecture models indicates that, rather than having a single one-size-fits-all model for reverse engineering, it may be more efficient and effective to use a combination of models for reverse engineering and complex binary analysis tasks.

C. Data and Input Representation

There is also some variation between the type of data input the various research papers provided the AI models with. For instance, both of the major decompilation tools, LLM4Decompile and SALT4Decompile, worked primarily with assembly code, with LLM4Decompile converting that assembly code to higher-level C code and SALT4Decompile creating Source-Level Abstract Logic Trees from assembly code as a way to more effectively layout the code's original

semantic structure. The compiler provenance identification paper takes a completely different approach by transforming provided binaries into grayscale images for processing by Vision Transformers. The benchmark papers primarily use binaries as their input for training data. Decompile-Bench was trained with over two million binary-source code pair; REBench used actual stripped binaries for its input data. These decisions about input representation show the importance of selecting the appropriate kind of training data for the task that needs to be solved, once again limiting the usefulness of a one-size-fits-all approach to reverse engineering tasks.

D. Limitations

In spite of all the success and progress made, the surveyed papers identify several significant limitations that need to be addressed. A significant problem with large language models, even outside of a reverse engineering context, is that they always run the risk of hallucinating false outputs, especially when analyzing unfamiliar architectures or highly optimized binaries. Many proposed systems are trained using datasets based on x86-64 Linux binaries, impacting their ability to extend to other architectures, operating systems, and programming languages. Several studies also note that compiler optimizations, binary stripping, and code obfuscation continue to limit model accuracy by removing valuable semantic information. These challenges suggest that much improvement in training methodology, dataset construction, and hybrid analysis techniques is still necessary for AI systems to consistently perform many complex reverse engineering tasks with a high degree of accuracy.

E. Overall Findings and Insights

To summarize, these surveyed papers cover a wide array of reverse engineering topics as they pertain to the usage and benefit of AI. They demonstrate that using AI to assist with reverse engineering tasks is becoming increasingly practical and effective as the AI systems continue to develop and more research is done on reverse engineering topics as a whole. There has been moderate success using consumer-level AI models like GPT-4, but they are limited by their lack of specialization, making these types of models a ”jack of all trades, master of none” when it comes to reverse engineering applications. However, models that are more specialized towards reverse engineering, such as ReCopilot, LLM4Decompile, and SALT4Decompile, have been shown to be significantly more successful at their respective tasks, including binary analysis and decompilation. Simultaneously, much work has been put into making sure benchmarking and evaluation tools are advancing at a similar rate to the actual analysis tools. The developers behind, REBench, BinMetric, and Decompile-Bench are aware of the need for standardized benchmarking tools to measure the future progress of other approaches to reverse engineering. Overall, there is much headway being made in the way of AI-assisted reverse engineering as developers move from purely theoretical research on the topic towards actual software implementation by combining AI tools with traditional reverse engineering approaches, as well as standardized benchmarking and evaluation metrics in the field.

IV. Future Work

There has been a lot of progress made in improving the abilities and use case of AI-assisted reverse engineering, yet there are still improvements that should be made before these techniques can become practical for real-world analysis. The surveyed papers identify several of these limitations, specifically as they pertain to model generalization, benchmark quality, and semantic reasoning. These issues will need to be addressed as the next stage of research on the topic of using AI in reverse engineering progresses.

One important change would be to improve generalization across architectures, programming languages, and compiler toolchains. Because so many of the models discussed in the survey were trained on x86-64 Linux binaries, they lack the ability to be generalized to software compiled under other systems such as ARM or embedded systems. Additionally, models that deal directly with source code should be trained on various programming languages, including multiple higher-level languages besides just C code. As software develops and becomes increasingly diverse, there is a greater necessity for models to be trained on data of varying operating systems, architectures, and programming languages.

One area that shows great potential if research is properly devoted to it in the future is that of hybrid-analysis reverse engineering techniques. Several surveyed papers reference the potential for combining LLMs with more traditional reverse engineering techniques and software, such as Ghidra. These papers show that static analysis, control-flow graphs, data-flow analysis, and abstract logic trees can provide critical data that significantly improves LLM performance. As with many areas of life, the future of LLMs will likely involve mixing their usage with that of traditional techniques to obtain the most thorough and comprehensive results realistically possible.

Several of the surveyed papers also discuss the need for improved benchmarks and evaluation methodologies. Although REBench, BinMetric, and Decompile-Bench indicate an awareness of the need for standardized evaluation and significant progress in that regard, more research needs to be done to include more complex data in future benchmarks, such as real-world malware, heavily obfuscated software, and binaries produced by diverse compiler configurations. The more robust and standardized benchmark and evaluation methodologies become, the easier it will be to compare the results of future research on reverse engineering and make

decisions about potential improvements.

Finally, future research should address the reliability and trustworthiness of AI-assisted reverse engineering systems. AI models in their current stage are notorious for providing questionable output, often simply lying to the user about the results of a given prompt. Hallucinated outputs, incorrect interpretations of provided code, and limited reasoning capabilities can make AI models a risky option to implement for any purpose, especially one as technical as reverse engineering. Because of this, more research needs to be done to improve models' reasoning, decrease the likelihood of hallucinations, and enable seamless collaboration with human developers and analysts. This will allow them to be better integrated into existing technologies and better benefit the field of reverse engineering.

V. Conclusion

Artificial intelligence has rapidly become one of the most involved areas of research in reverse engineering. The surveyed papers in this work demonstrate substantial progress in applying large language models to tasks such as binary decompilation, semantic recovery, vulnerability analysis, and compiler provenance identification. In addition to introducing very successful AI models, recent research has also emphasized the importance of standardized benchmark and evaluation frameworks to consistently compare and measure reverse engineering systems.

There are several common trends in this survey. Firstly, many researchers are moving away from evaluating general-purpose language models and instead developing specialized models trained specifically for binary analysis tasks, partially due to recent advances in AI technology. Similarly, there is an significant emphasis on using program analysis techniques, such as control-flow analysis, data-flow information, and abstract logic representations, in combination with LLMs to improve semantic understanding of analyzed binaries. Additionally, benchmark projects such as REBench, Decompile-Bench, and BinMetric are helping create consistent evaluation standards for future research.

Although current AI-assisted reverse engineering systems have been very successful, several major challenges remain. Limited support for diverse architectures, dependence on large training datasets, hallucinated outputs, and reduced performance on optimized binaries make it difficult for many of these tested systems to become widely adopted. Addressing these limitations by improving model architectures, using more robust datasets, utilizing hybrid analysis techniques, and having more comprehensive evaluation methods opens the doors to future research.

Overall, the surveyed works indicate that AI-assisted reverse engineering is starting to become less theoretical and more practical in nature. As language models continue to improve and start to be combined with traditional binary analysis techniques, they are likely to become very useful tools for cybersecurity professionals, malware analysts, and software engineers. Continued development of these AI-assisted reverse engineering systems will certainly allow them to become more accurate, reliable, and useful to software analysts.

References

[1] G. Chen et al., "Recopilot: Reverse engineering copilot in binary analysis," arXiv.org, https://arxiv.org/abs/2505.16366 (accessed May 28, 2026).

[2] H. Tan, Q. Luo, J. Li, and Y. Zhang, "LLM4Decompile: Decompiling binary code with large language models," arXiv.org, https://arxiv.org/abs/2403.05286 (accessed May 28, 2026).

[3] J. Y. Won, X. Jin, S. Ma, and Z. Lin, "Rebench: A procedural, fair-by-construction benchmark for LLMS on stripped-binary types and names (extended version)," arXiv.org, https://arxiv.org/abs/2604.27319 (accessed May 28, 2026).

[4] Y. Wang, X. Xu, X. Zhu, X. Gu, and B. Shen, "SALT4Decompile: Inferring source-level abstract logic tree for LLM-based Binary Decompilation," arXiv.org, https://arxiv.org/abs/2509.14646 (accessed May 28, 2026).

[5] H. Tan et al., "Decompile-bench: Million-scale binary-source function pairs for real-world binary decompilation," arXiv.org, https://arxiv.org/abs/2505.12668 (accessed May 28, 2026).

[6] X. Shang, G. Chen, S. Cheng, and B. Wu, "Binmetric: A comprehensive binary code analysis benchmark for large," ijcai.org, https://www.ijcai.org/proceedings/2025/0858.pdf (accessed May 29, 2026).

[7] X. Zhu et al., "When software security meets large language models: A survey," IEEE/CAA Journal of Automatica Sinica, https://www.ieee-jas.com/article/doi/10.1109/JAS.2024.124971 (accessed May 28, 2026).

[8] A. Adhikari and P. Kulkarni, "Survey of techniques to detect common weaknesses in program binaries - sciencedirect," ScienceDirect.com, https://www.sciencedirect.com/science/article/pii/S2772918424000274 (accessed May 29, 2026).

[9] S. Pordanesh and B. Tan, "Exploring the efficacy of large language models (GPT-4) in binary reverse engineering," arXiv.org, https://arxiv.org/abs/2406.06637 (accessed May 28, 2026).

[10] W. Khan, "Compiler-provenance identification in obfuscated binaries using vision transformers - sciencedirect," Compiler-provenance identification in obfuscated binaries using vision transformers, https://www.sciencedirect.com/science/article/pii/S2666281724000830 (accessed May 29, 2026).